Predictive Deadlock Warning: Early Detection of Unsafe States from Resource-Request Patterns

Revanth Reddy Lingala and Padigela Srujan
Department of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad, India

***Abstract*—Banker's algorithm tells an operating system whether the current resource-allocation state is *safe*; it gives no signal that the system is drifting towards an unsafe state. We ask whether simple, explainable indicators computed from the allocation state and the recent request stream can raise an alarm *before* Banker's algorithm reports an unsafe state, and how early and how reliably. We design a rule-based (no machine learning) early-warning algorithm that combines five indicators—resource scarcity, need pressure, contention, waiting ratio and request rate—into a 0–100 risk score with a persistence (k) and refractory rule. A purpose-built observer-mode simulator grants every valid request, so the system can actually become unsafe, and records Banker's safety and deadlock-detection outcomes at every step. We evaluate with event-based metrics from the early-warning literature (alarm precision, event recall, lead time, censoring and refractory periods) against explicit baselines, tuning on one seed range and reporting on a disjoint one. Over 300,000 simulated sequences (676.2 million requests, each followed by a Banker's safety check) across three workload scenarios and 5 system sizes up to 200 processes, the combined score reached F1 = 0.28 [0.28, 0.28] with a median warning 10 steps ahead of the unsafe onset, versus 0.16 for a free-resource threshold and 0.01 for random alarms at a matched rate. A single indicator, contention, scored 0.34. An exact one-step lookahead baseline scored 0.74 on the sequences where it was computed, with similar lead times on the events both methods caught, but cost 39 times as much computation as the whole simulation at 200 processes. Unsafe episodes almost always ended in deadlock when processes eventually request their full claims. Cheap indicators therefore give useful but moderate early warning at negligible cost, while exact lookahead is far more accurate at a much higher cost.**

***Index Terms*—deadlock, Banker's algorithm, early warning, resource allocation, operating systems, simulation**

I. Introduction

Deadlock handling is a core operating-system concern: processes that hold resources while waiting for others can block each other forever. The classical avoidance method, Banker's algorithm [1], [2], checks whether a state is *safe*—whether some ordering lets every process obtain its maximum remaining claim and finish. The check is binary and instantaneous: it says *safe now* or *unsafe now*, but nothing about whether the system is heading towards trouble.

This paper studies a complementary question, analogous to a smoke detector that sounds before the fire alarm: *can simple signals warn that an unsafe state is coming, before Banker's algorithm detects it?* We address seven research questions:

- RQ1: Do indicators computed from resource-request patterns give early warning of an unsafe state?
- RQ2: How much lead time do alarms provide?
- RQ3: What alarm precision and event recall do they achieve?
- RQ4: Which indicators contribute most, and does the combined score beat simple baselines?
- RQ5: How does warning quality vary across workload scenarios?
- RQ6: How often do unsafe states become deadlocks, and can alarms anticipate deadlocks?
- RQ7: How does runtime scale with processes and resources?

Fig. 1 shows the idea on three test sequences: the risk score rises, an alarm fires, and the system turns unsafe and then deadlocks some steps later.

Our contributions are: (i) a fully explainable, rule-based early-warning algorithm with no trained model; (ii) an observer-mode simulator with three controlled workload scenarios; (iii) an event-based evaluation protocol borrowed from clinical and earthquake early warning [3], [4], with censoring, refractory periods and strict lead-time rules; (iv) explicit baselines including random alarms at a matched rate and an exact one-step lookahead; (v) separate measurement of unsafe and deadlocked outcomes; and (vi) a scaling study, with all experiments run headless on AWS EC2.

II. Background

A. System model and Banker's algorithm

A system has m resource types with total_j units each and n processes. Process i declares a maximum claim Max_{ij}, holds Alloc_{ij} and may still request $\mathit{Need}_{ij} = \mathit{Max}_{ij} - \mathit{Alloc}_{ij}$. Free units are $\mathit{Avail}_j = \mathit{total}_j - \sum_i \mathit{Alloc}_{ij}$. Banker's safety check [1], [2] repeatedly picks an unfinished process whose need fits the working vector, lets it finish and returns its allocation; the state is safe if every process can finish. The standard algorithm costs $\Theta(mn^2)$ time [5]; amortised $O(mn)$ variants [6] and the dynamic DBDAA algorithm [7] reduce this.

B. Unsafe is not deadlock

Banker's check is pessimistic: it assumes every process may request its full remaining claim. Deadlock detection is optimistic: it assumes running processes finish and release everything, so only blocked processes' pending requests matter [2], [8]. A state can be safe and not deadlocked, unsafe and not deadlocked, or unsafe and deadlocked, but never safe and deadlocked [8]. We therefore record both outcomes at every

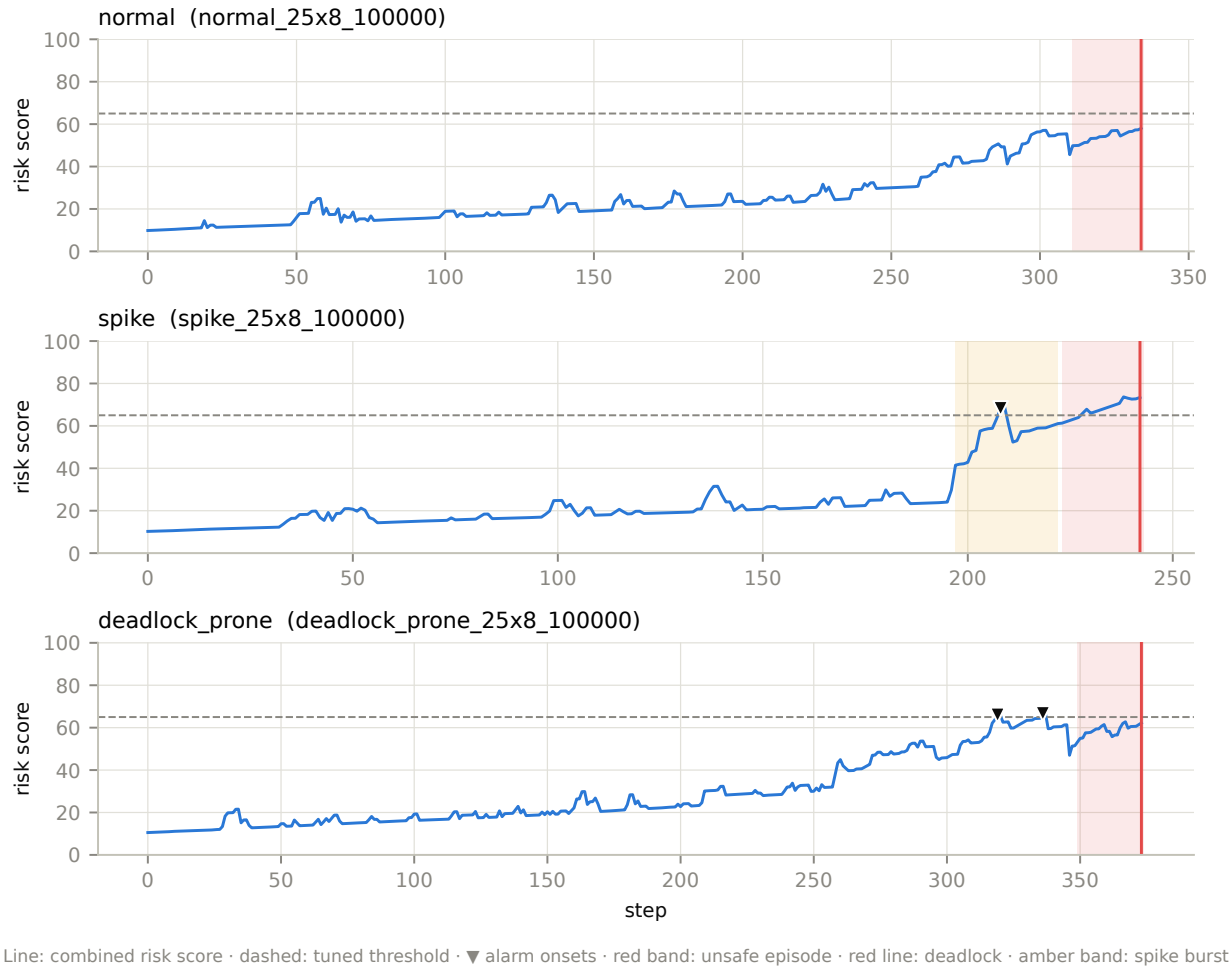


Fig. 1. Example test sequences (25 processes, 8 resource types), one per scenario. Blue: combined risk score; dashed: tuned threshold; triangles: alarm onsets; red band: unsafe episode; red line: deadlock; amber band: spike burst.

step and use the unsafe onset as the primary prediction target and the deadlock onset as a secondary one.

III. Related Work

Faster safety checks [6], [7] reduce the cost of answering "safe now?" but do not anticipate danger. Deadlock *prediction* has been studied with machine learning: Le Ngoc and Tran Cong used linear regression to forecast deadlock possibility for virtual machines in CloudSim [9], and reinforcement-learning and AI-based predictors have been proposed for thread deadlocks [10]. Prediction is therefore not new at field level. Our work differs in three ways: the warning rule is fully explainable and untrained; scenarios are controlled so that gradual and sudden build-ups can be compared; and evaluation follows event-based early-warning practice—alarm precision and event recall [3], warnings at or after onset counted as misses [4], refractory periods and random baselines at a matched alarm rate—which deadlock studies rarely apply.

IV. Simulator

A. Observer mode

If Banker's algorithm were used to *refuse* unsafe requests, the system would never become unsafe and nothing could be predicted. Our simulator therefore grants every valid request and uses Banker's algorithm only to *observe*. Each step: (1) waiting processes are retried first-come first-served, and any process whose claim is now met finishes and releases everything, repeating until nothing changes; (2) one active, non-waiting process issues a non-zero request no larger than its need—it is granted if it fits the free units and otherwise the process blocks with the request pending; (3) completed processes release their resources; (4) in the *normal* scenario a process may voluntarily release part of its allocation; (5) the simulator records whether the state is unsafe, whether it is deadlocked, and the five indicators. A sequence ends at the first deadlock, when all processes finish, or at a step cap.

B. Correctness

Initial states are regenerated until safe. Invariants (conservation of units, allocation $\leq$ claim, pending $\leq$ need, finished processes hold nothing) are asserted in a debug mode used by the test suite. A vectorised NumPy safety check, which finishes every process that fits in each pass, is verified against the textbook loop on 2,000 random states, as is deadlock detection, together with the textbook examples of [2]. The property *deadlocked* $\Rightarrow$ *unsafe* holds on all random states tested. Because detection assumes non-waiting processes release everything, grants to, releases by and completions of non-waiting processes cannot change the detection result; detection therefore reruns only when the waiting set or a pending request changes, which we verified against every-step detection. In total 122 unit and property tests cover every row of our edge-case matrix (system states, requests, life-cycle, indicators, alarm logic, evaluation and experiment hygiene).

C. Workload scenarios

Requests are fractions of remaining need, so behaviour is comparable across sizes. The main calibration parameter is claim tightness, $\sum_i \mathit{Max}_{ij}/\mathit{total}_j$; initial states draw it from a scenario-specific range. *Normal*: small single-resource requests and occasional partial releases. *Spike*: normal traffic, then—once a random 20–60% of the initial total need has been requested—80% of processes make broad grabs of 30–60% of their remaining need, capped by free units, holding back the last unit. *Deadlock-prone*: process i prefers resource $(i{+}1)$ mod m and rarely takes its final unit. Parameters were calibrated on tuning seeds only so that each scenario yields a mix of sequences with and without unsafe events (35%, 60% and 62% of test sequences contain at least one, averaged over sizes).

V. Early-Warning Algorithm

After every step five indicators in $[0, 1]$ are computed: *scarcity* $= 1 - \sum_j \mathit{Avail}_j / \sum_j \mathit{total}_j$; *need pressure* $= \frac{1}{m} \sum_j S_j/(S_j{+}\mathit{Avail}_j)$, where S_j is the total outstanding need of active processes; *contention*, the fraction of resource types for which at least two active processes need more than is free; *waiting ratio*, the fraction of active processes that are blocked; and *request rate*, the clipped relative increase in units requested over the last $W{=}10$ steps compared with the previous 10. The risk score is the weighted mean $100 \sum_i w_i x_i / \sum_i w_i$. An *alarm onset* occurs at step t if the score has been at least the threshold for k consecutive steps ending at t and no onset occurred in the previous R steps (refractory period). Alarms are suppressed while the system is already unsafe, since Banker's algorithm is then already signalling. The tuned combined score uses threshold 65, $k = 1$, $R = 10$ and weights "double_contention".

VI. Evaluation Method

A. Events and metrics

An *unsafe onset* is a step where the state becomes unsafe; onsets after fewer than five safe steps are merged into the previous episode. An alarm at step a is *true* if an onset falls in $[a{+}1, a{+}H_{\max}]$ ($H_{\max} = 20$); an onset is *captured* by the earliest unused alarm in its window, giving its lead time. An alarm at the onset step is a miss (detection, not prediction). Alarms in the final $H_{\max}$ steps of a sequence cut by the step cap are *censored* and excluded, whereas in sequences that ended by deadlock or completion the future is known and such alarms count as false. Onsets in the first three steps are too early for any method and are excluded for all methods. We report micro-averaged alarm precision, event recall, F1, false alarms per 1,000 steps, lead-time distributions and 95% bootstrap confidence intervals (1,000 resamples of sequences, stratified by scenario and size).

B. Baselines

Availability raises an alarm when the free fraction of units falls below a threshold; it is identical to the scarcity indicator alone. Each other indicator is also evaluated alone. *Random* raises alarms independently at each eligible step with a rate matched, on the tuning split, to the number of alarms of the tuned combined score. *Lookahead* is a strong, expensive reference: the fraction of running processes for which granting the largest feasible request would make the state unsafe; it costs one safety check per process per step and is therefore computed for all sequences with up to 25 processes and for a subset of seeds for larger systems.

C. Tuning and data split

Every method's parameters (threshold 5–95 in steps of 5, $k \in \{1, 2, 3\}$, $R \in \{10, 20\}$, and for the combined score eleven weight vectors: equal, each indicator doubled, each removed) are chosen on tuning seeds to maximise F1 subject to at most 5 false alarms per 1,000 steps and a median lead of at least 3 steps. All reported numbers come from a disjoint test-seed range.

D. Experimental setup

Each of the three scenarios was run at 5 system sizes (10–200 processes, 5–20 resource types) with 10,000 seeds per cell for both the tuning and the test split: 150,000 tuning and 150,000 test sequences, 676.2 million simulated requests in total (Table I). Experiments ran headless on one AWS EC2 c7i.8xlarge instance (32 vCPUs, 62 GB RAM, region ap-south-2) with Python, NumPy and a process pool; the complete pipeline—tests, calibration, 300,000 simulations, tuning, evaluation, scaling and figures—took 2 h 43 min and about $4.11 of compute. Sequences were written as chunked Parquet files so interrupted runs resume, and every run records its configuration hash, code version and machine (commit `d3de8e5`).

VII. Results

A. RQ1–RQ3: early warning, lead time, precision and recall

Table II gives the main results. The combined score achieved F1 = 0.28 (precision 0.24, recall 0.34) with 0.33 false alarms per 1,000 steps, against 0.01 for random alarms at the same

TABLE I

EXPERIMENTAL SETUP PER CELL (TEST SPLIT). TEST SPLIT: 150,000 SEQUENCES, 338,262,425 STEPS; TUNING SPLIT: 150,000 SEQUENCES, 337,917,758 STEPS. MACHINE: LINUX-7.0.0-1014-AWS-X86_64-WITH-GLIBC2.39 (32 CPUS, C7I.8XLARGE).

scenario	n	m	seqs	steps	events	% with event	% deadlock	tightness
normal	10	5	10,000	1,071,547	2,754	26.90	25.40	1.43
normal	25	8	10,000	4,494,155	1,674	15.70	14.30	1.43
normal	50	10	10,000	11,527,074	1,270	11.30	10.10	1.43
normal	100	20	10,000	39,280,274	8,469	65.80	62.70	1.43
normal	200	20	10,000	82,123,372	7,358	56.20	53.60	1.43
spike	10	5	10,000	707,662	7,066	70.70	70.70	1.58
spike	25	8	10,000	2,939,715	5,954	59.50	59.50	1.58
spike	50	10	10,000	7,434,425	5,317	53.20	53.20	1.58
spike	100	20	10,000	27,023,589	6,185	61.90	61.90	1.58
spike	200	20	10,000	56,973,051	5,346	53.50	53.50	1.58
deadlock_prone	10	5	10,000	1,020,230	4,073	40.70	40.70	1.48
deadlock_prone	25	8	10,000	4,036,769	4,047	40.50	40.50	1.48
deadlock_prone	50	10	10,000	9,877,410	4,513	45.10	45.10	1.48
deadlock_prone	100	20	10,000	29,793,804	9,316	93.20	93.20	1.48
deadlock_prone	200	20	10,000	59,959,348	9,270	92.70	92.70	1.48

TABLE II

MAIN RESULTS, UNSAFE TARGET, $H_{max} = 20$, TEST SPLIT, 95% BOOTSTRAP CIS.

method	precision	recall	F1	FA/1000	lead med.	lead≥5	alarms
combined	0.24 [0.23, 0.24]	0.34 [0.34, 0.35]	0.28 [0.28, 0.28]	0.33	10	0.75	147,529
need_pressure	0.31 [0.31, 0.32]	0.14 [0.13, 0.14]	0.19 [0.19, 0.19]	0.07	7	0.66	35,750
contention	0.30 [0.30, 0.30]	0.39 [0.38, 0.39]	0.34 [0.34, 0.34]	0.22	7	0.69	106,873
waiting_ratio	0.24 [0.24, 0.24]	0.26 [0.26, 0.26]	0.25 [0.25, 0.25]	0.31	13	0.86	136,824
request_rate	0.16 [0.16, 0.16]	0.12 [0.11, 0.12]	0.14 [0.13, 0.14]	0.15	9	0.75	59,822
availability	0.11 [0.10, 0.11]	0.34 [0.34, 0.35]	0.16 [0.16, 0.16]	0.71	8	0.72	268,034
lookahead	0.69 [0.68, 0.70]	0.81 [0.81, 0.81]	0.74 [0.74, 0.75]	0.62	4	0.48	35,744
random	0.00 [0.00, 0.01]	0.01 [0.01, 0.01]	0.01 [0.01, 0.01]	0.43	11	0.82	147,855

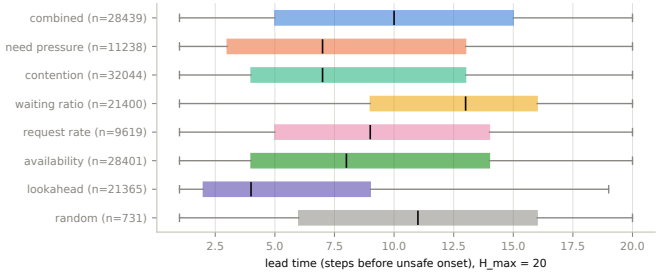


Fig. 2. Lead times of captured unsafe onsets ($H_{\max} = 20$).

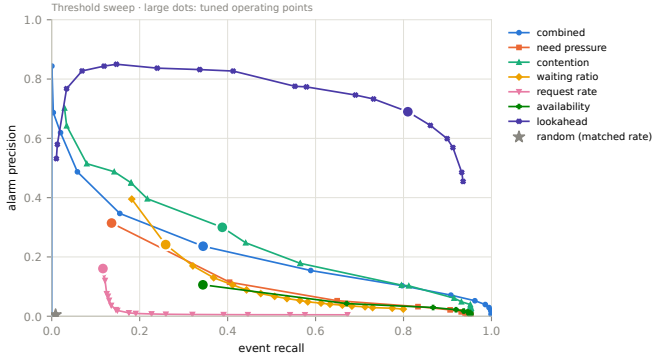


Fig. 3. Precision–recall curves (threshold sweep at the tuned k, R and weights; plotted on test data, never used for selection). Large dots: tuned operating points; star: random baseline.

rate: indicators clearly carry predictive signal (RQ1). Captured events were warned a median of 10 steps ahead and 0.75 of them at least five steps ahead (Fig. 2; RQ2). Precision and recall are moderate (RQ3); Fig. 3 shows the full precision–recall trade-off.

B. RQ4: which indicators matter

Contention alone (F1 = 0.34) scored higher than the combined score, while need pressure (0.19), waiting ratio (0.25), request rate (0.14) and availability (0.16) scored lower. The tuner's best combination doubled the weight of contention ("double_contention"). The ablation (Table III) shows that removing contention or request rate hurts the equal-weight combination most, while removing waiting ratio helps: averaging weaker indicators dilutes the strongest one, an argument for selecting indicators rather than combining all of them. On the sequences where it was computed, the exact lookahead baseline reached F1 = 0.74 against 0.44 for the combined score (Table IV). On the 8,012 events captured by both, lead times were similar (median 5 vs. 4 steps; the combined score was earlier for 48%), so lookahead's advantage is accuracy, bought at 39× the simulation's own compute at 200 processes. Against availability, the combined score warned earlier on jointly captured events (median 11 vs. 8 steps).

TABLE III
ABLATION: COMBINED SCORE WITH ONE INDICATOR REMOVED (EACH RE-TUNED ON THE TUNING SPLIT).

variant	precision	recall	F1	FA/1000
all five, equal	0.17	0.40	0.24	0.59
without scarcity	0.19	0.39	0.26	0.47
without need_pressure	0.17	0.45	0.25	0.64
without contention	0.14	0.36	0.20	0.65
without waiting_ratio	0.23	0.31	0.27	0.30
without request_rate	0.15	0.41	0.22	0.93

TABLE IV
ALL METHODS ON THE SEQUENCES WHERE THE LOOKAHEAD BASELINE WAS COMPUTED.

method	precision	recall	F1	lead med.
combined	0.56	0.37	0.44 [0.44, 0.45]	5
need_pressure	0.65	0.11	0.18 [0.18, 0.19]	4
contention	0.46	0.59	0.51 [0.51, 0.52]	6
waiting_ratio	0.32	0.09	0.14 [0.14, 0.15]	4
request_rate	0.44	0.27	0.33 [0.33, 0.34]	8
availability	0.48	0.22	0.30 [0.29, 0.30]	4
lookahead	0.69	0.81	0.74 [0.74, 0.75]	4
random	0.03	0.01	0.01 [0.01, 0.01]	9

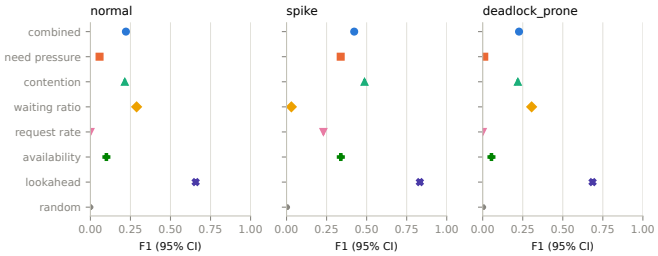


Fig. 4. F1 per scenario with 95% bootstrap confidence intervals.

TABLE V
F1 PER SCENARIO (UNSAFE TARGET, $H_{max} = 20$).

method	deadlock_prone	spike	normal
combined	0.23	0.42	0.22
need_pressure	0.01	0.34	0.06
contention	0.22	0.49	0.21
waiting_ratio	0.31	0.03	0.29
request_rate	0.00	0.23	0.00
availability	0.05	0.34	0.10
lookahead	0.69	0.83	0.66
random	0.01	0.01	0.00

C. RQ5: scenarios

Warning quality depends strongly on how danger builds (Fig. 4, Table V). Spike workloads were easiest (combined F1 = 0.42): bursts raise several indicators at once, and unsafe onsets followed a median 26 steps after the burst began (only 3.0% occurred before it). Gradual build-ups were hardest: F1 = 0.22 for normal and 0.23 for deadlock-prone workloads. There, waiting ratio alone (0.29, 0.31) did better than the combination, whereas contention's overall lead comes mainly from spikes (0.49 vs. 0.42).

D. RQ6: unsafe versus deadlock

In our model unsafe episodes rarely recover: 100% of resolved episodes ended in deadlock under spike and 100% under deadlock-prone workloads, and 77% under normal workloads, where voluntary releases allow some recovery (Fig. 5). This is expected: processes eventually request their full claims, so Banker's worst-case assumption is realised. For the deadlock target, alarms are not suppressed during unsafe periods; waiting ratio is then the best predictor at $H_{\max} = 20$ (F1 = 0.48, above lookahead's 0.37 on its subset; Table VI), because blocked processes accumulate as a deadlock forms. With a 50-step horizon lookahead leads (0.76 vs. 0.65).

E. Sensitivity to the warning horizon

With the same alarms, widening the horizon raises F1 for every method (combined: 0.11, 0.19, 0.28 and 0.40 for $H_{\max}$ = 5, 10, 20, 50; Fig. 6), showing that many alarms precede onsets by more than 20 steps.

F. RQ7: runtime scaling

The vectorised safety check was 1.3× (10 processes) to 7.7× (200 processes) faster than the textbook loop (Table VII, Fig. 7). Measured growth with n was far below the $\Theta(mn^2)$ worst case (log–log slopes 0.29 vectorised and 0.77 reference at $m = 20$): realistic states clear in a few passes, and NumPy overhead dominates at these sizes. Event-driven deadlock detection ran on only a small fraction of steps.

VIII. DISCUSSION

Circularity. Indicators and Banker's check read the same state, so some correlation is built in; this is why we include availability, random and lookahead baselines. Beating random by a wide margin and availability by a clear margin shows the indicators add information; the lookahead baseline shows how much an exact—but expensive—state-based test can add.

A trade-off, not a winner. Cheap indicators cost almost nothing per step but are only moderately precise; lookahead is far more precise, with similar lead times, but costs one safety check per running process per step (39× the simulation's compute at 200 processes). A practical design could use cheap indicators as a first stage and run lookahead only when the score is elevated.

Where alarms fail. Gradual build-ups give little distinctive signal before the boundary is crossed; spikes are easier because several indicators move together.

IX. THREATS TO VALIDITY

Workloads are synthetic and processes declare maximum claims, as Banker's algorithm requires; real systems rarely know claims. The process set is fixed (no arrivals). Claim tightness, which largely determines whether a scenario becomes unsafe, was calibrated to give class balance. Flicker merging, the warm-up exclusion and ending a sequence at the first (possibly partial) deadlock are modelling choices. The lookahead baseline was computed on a subset of seeds

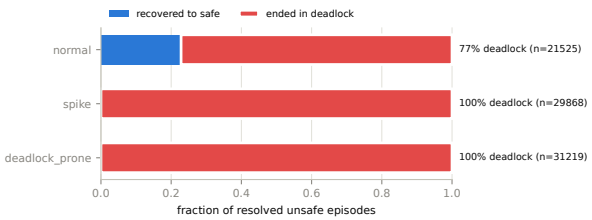


Fig. 5. Outcome of resolved unsafe episodes per scenario.

TABLE VI
DEADLOCK-ONSET TARGET ($H_{max} = 20$, ALARMS NOT MASKED WHILE UNSAFE).

method	precision	recall	F1	lead med.	events
combined	0.21	0.71	0.33	13	77,700
need_pressure	0.18	0.34	0.23	10	77,700
contention	0.17	0.54	0.26	12	77,700
waiting_ratio	0.33	0.83	0.48	12	77,700
request_rate	0.07	0.06	0.06	15	77,700
availability	0.10	0.52	0.16	10	77,700
lookahead	0.32	0.43	0.37	15	25,862
random	0.00	0.01	0.01	10	77,700

TABLE VII
RUNTIME SCALING: TIME PER BANKER'S SAFETY CHECK ON RANDOM VALID STATES (VECTORISED VS. REFERENCE) AND WHOLE-SIMULATOR TIME PER STEP.

n	m	vec μs	ref μs	speed-up	μs/step
10	5	7.1	9.5	1.3	96.7
10	10	6.6	9.0	1.4	99.3
10	20	6.7	10.2	1.5	98.8
25	5	8.7	18.5	2.1	97.4
25	10	7.1	15.9	2.2	102.6
25	20	7.3	16.6	2.3	105.0
50	5	8.8	29.9	3.4	99.2
50	10	7.9	27.9	3.5	109.6
50	20	8.7	28.7	3.3	116.1
100	5	11.7	60.1	5.1	106.9
100	10	9.7	47.5	4.9	115.0
100	20	10.9	52.3	4.8	114.9
200	5	17.3	133.9	7.7	125.0
200	10	13.6	104.6	7.7	129.0
200	20	16.3	101.4	6.2	139.6

for larger systems, so its comparison uses that subset. Three single-indicator baselines (need pressure, request rate, availability) selected the highest threshold in the tuning grid, so slightly better settings may exist for them.

X. CONCLUSION AND FUTURE WORK

Simple, explainable indicators can warn of an approaching unsafe state several steps before Banker's algorithm detects it, at a clearly better-than-chance and better-than-availability level, with the strongest gains for sudden workload changes. Exact lookahead is much more accurate, with similar lead times, but far more expensive. Future work: learned models on the same data, real traces, incremental safety checks [7], two-stage warning, and an avoidance-mode variant that acts on warnings.

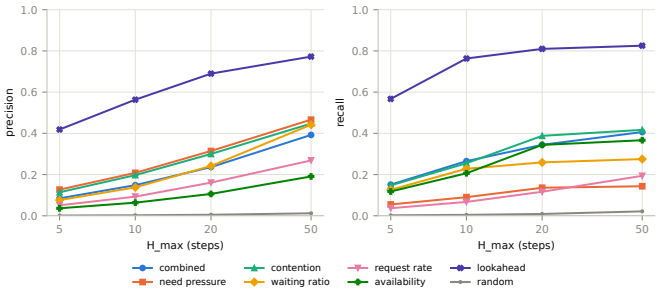


Fig. 6. Precision and recall as the warning horizon $H_{\max}$ varies (same alarms).

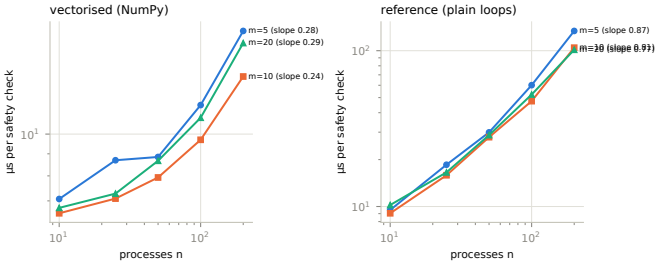


Fig. 7. Time per safety check versus number of processes (log–log), vectorised and reference implementations.

REFERENCES

[1] E. W. Dijkstra, "Een algorithme ter voorkoming van de dodelijke omarming," Technische Hogeschool Eindhoven, Tech. Rep. EWD108, 1965. [Online]. Available: https://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley.

[3] S. L. Hyland *et al.*, "Early prediction of circulatory failure in the intensive care unit using machine learning," *Nature Medicine*, 2020.

[4] J. Münchmeyer *et al.*, "The transformer earthquake alerting model: A new versatile approach to earthquake early warning," arXiv:2009.06316, 2020.

[5] MIT Algorithm Wiki, "Banker's algorithm (deadlock avoidance)," https://algorithm-wiki.csail.mit.edu/wiki/Banker%27s_Algorithm_(Deadlock_Avoidance_Deadlock_avoidance), accessed October 2026.

[6] "A modified banker's algorithm with amortised $o(mn)$ complexity," *BIT Numerical Mathematics*, bibliographic details to be verified against the DOI.

[7] M. F. Zohora, F. Farhin, and M. S. Kaiser, "DBDAA: A real-time approach to dynamic banker's deadlock avoidance algorithm with optimized time complexity," *PLOS ONE*, 2024.

[8] A. Gottlieb, "Operating systems lecture notes: deadlock avoidance, safe and unsafe states," New York University, https://cs.nyu.edu/~gottlieb/courses/2000s/2004-05-spring/os202/lectures/lecture-13.html.

[9] H. Le Ngoc and H. Tran Cong, "Enhancing load balancing in cloud computing through deadlock prediction," in *Industrial Networks and Intelligent Systems (INISCOM 2023)*. Springer, 2023.

[10] J. Oluwaseyi, "AI-enhanced predictive systems for thread deadlock resolution," EasyChair Preprint 14955, 2024.